

DPI343

Computer Organization and Assembly Language

Lecture 6

Lectures - prof. em. U. Sukovskis

Labs for local students - assoc. prof. P. Rusakovs

Labs for foreign students - assoc. prof. G. Alksnis

Autumn 2025



RTU
DATORZINĀTNES,
INFORMĀCIJAS
TEHNOLOĢIJAS
UN ENERĢĒTIKAS
FAKULTĀTE

Procedures

Parameter passing



Procedures in Assembly language

```
SimplestP PROC  
    ret  
SimplestP ENDP
```

The simplest procedure, that does nothing but returns

```
Name1 PROC  
    :::  
    ret  
Name1 ENDP
```

Instructions of the procedure

```
Name3 PROC  
    :::  
    call SimplestP  
    :::  
    call Name1  
    :::  
    ret  
Name3 ENDP
```

Transfers control to the entry point of the procedure *SimplestP*

Transfers control to the entry point of the procedure *Name1*

Instructions CALL and RET

Simple explanation

CALL *label*

Execution:

- 1) Address of the next instruction that follows the CALL instruction (i.e. return address) is stored onto top of the stack (*just like the instruction push does*)
- 2) Control is transferred to the *label* (i.e. the EIP register is loaded with the address specified)

RET

Execution:

- 1) Address is retrieved from the top of the stack (*like the pop instruction*)
- 2) Control is transferred to this address (i.e. the EIP register is loaded with the address from the top of the stack)

Execution of these instructions is similar to unconditional jump.

Instructions CALL and RET

Assembly supports near and far procedures.

Near calls and returns transfer control between procedures in the same code segment.

Far calls and returns can transfer control between different code segments.

```
ProcName1 PROC near  
    :::  
    ret  
ProcName1 ENDP
```

```
ProcName2 PROC far  
    :::  
    ret  
ProcName2 ENDP
```

In fact, when working in 32-bit flat memory mode, there is no need to use far procedures, because all the code is in one memory segment. We will return to this issue when it comes to writing programs for real address mode.

Instructions CALL and RET

More detailed explanation

CALL *label*

- 1) in the case of **FAR** procedure value of the code segment register CS is pushed onto stack
- 2) in the case of **FAR** procedure CS is loaded with segment value of the address *label*
- 3) **value of the EIP register (instruction pointer which contains address of the next instruction that follows the CALL instruction) is pushed onto stack**
- 4) **offset value of the address specified by *label* is loaded into EIP register, i.e. control is transferred to the label**

RET [n]

- 1) **register EIP is loaded with value from the top of the program stack**
- 2) in the case of **FAR** procedure the CS register is loaded with value from the top of the program stack (like the operation pop cs)
- 3) **If optional operand n is provided then top of the program stack is adjusted: $ESP = ESP + n$
It is usually used to bypass parameters placed in the stack by the caller.**

The compiler will choose which version of CALL and RET to use in the executable code, based on the information in the directives that declare the procedure.

Passing parameters to procedures

Different approaches may be used:

- Using general purpose registers
- Using program stack
- Using global variables

In this course we will learn the basic techniques and principles. The full story is more complicated...

Passing parameters via registers

Example

The procedure Ones counts number of ones in the rightmost bits of a doubleword

Parameters: EAX - doubleword in which bits will be counted

ECX - number of rightmost bits to count

Return value: EAX contains number of bits with the value one

If values of parameters are

EAX = 29

ECX = 4

EAX = 0000001Dh = 00000000000000000000000000000000**11101**b

then the result should be

EAX = 3

How to check each bit of the EAX register?

You may use bitwise AND operation to check one low order bit of the register and then shift right the content of the register to check next bit, and so on

EAX = 29 = 1Dh = 0000001Dh

EAX = 0000'0000'0000'0000'0000'0000'0001'1101

AND 0000'0000'0000'0000'0000'0000'0000'0001

0000'0000'0000'0000'0000'0000'0000'0001

result == 0?

No: counter = counter + 1

Yes: shift right EAX by 1 bit

EAX = 0000'0000'0000'0000'0000'0000'0000'1110

AND 0000'0000'0000'0000'0000'0000'0000'0001

0000'0000'0000'0000'0000'0000'0000'0000

result == 0?

No: counter = counter + 1

Yes: shift right EAX by 1 bit

```
                ; ecx contains number of rightmost bits to count

                xor    ebx, ebx    ; assign 0 to EBX
testb: test    eax, 1            ; test the rightmost bit of EAX
                jz     skip       ; do not increment EBX if bit is 0
                inc    ebx        ; increment EBX
skip: shr     eax, 1             ; shift right content of EAX by 1 bit
                loop   testb      ; repeat
```

Passing parameters via registers

Example

;Procedure Ones - counts number of ones in the rightmost bits of a doubleword
;Parameters:EAX - doubleword in which bits will be counted
; ECX - number of the rightmost bits to count
;Return value: EAX contains number of bits with the value one

Ones proc near

```
push ebx      ; save ebx value
push ecx      ; save ecx value
```

```
xor ebx, ebx  ; counter ebx = 0
testb: test eax, 1 ; test the rightmost bit of EAX
      jz skip    ; skip if bit is zero
      inc ebx    ; otherwise increment counter EBX
skip: shr eax, 1 ; shift right EAX by 1 bit
      loop testb ; repeat
      mov eax, ebx ; copy result to EAX
```

```
pop ecx      ; restore register values (in reverse order!)
pop ebx
```

```
ret          ; return to the caller
```

Ones endp

The EAX register is used in the procedure. Why don't we save it on the stack and restore it like the EBX and ECX registers?

Passing parameters via registers

Example (cont.)

```
.data
```

```
num    dword 29
```

```
buff   byte 11 dup (' '), 0
```

```
.code
```

```
main   proc
```

```
    mov  eax, num    ; prepare parameter in eax
```

```
    mov  ecx, 4      ; prepare parameter in ecx
```

```
    call Ones        ; when return from the procedure, the result is in eax
```

```
; convert numeric value of the register eax to text for output
```

```
    push 10          ; 3rd parameter base of decimal system
```

```
    push offset buff ; 2nd parameter - address of the buffer
```

```
    push eax         ; 1st parameter - integer value to convert
```

```
    call _ltoa
```

```
    add  esp, 12      ; adjust top of the stack, because puts uses cdecl calling convention
```

```
; output to console
```

```
    push offset buff ; parameter - address of the zero terminated text
```

```
    call puts
```

```
    add  esp, 4       ; adjust top of the stack, because puts uses cdecl calling convention
```

```
; exit
```

```
    invoke ExitProcess@4, 0 ; compiler replaces invoke directive with
```

```
    ; two instructions - push and call
```

```
main   endp
```

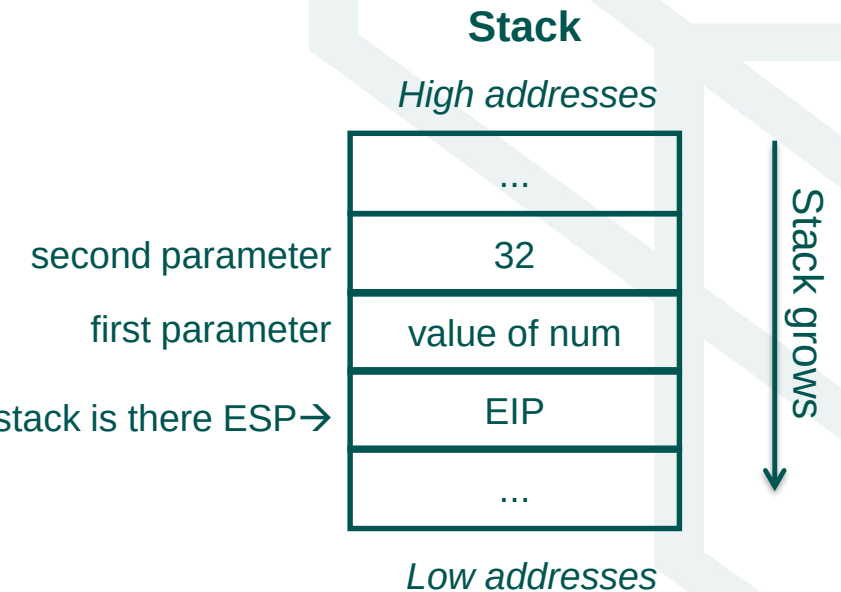
Passing parameters via stack

```
.data
num  dword  -2
buff byte  11 dup (' '), 0

.code
:::
    push 32      ; 2nd parameter
    push num     ; 1st parameter
    call Ones
:::

Ones proc near
    . . .
    ret 8
Ones endp
```

At the entry to procedure *Ones* top of the stack is there ESP→



Standard calling convention (stdcall) requires that arguments are pushed into stack from right to left (i.e. the last parameter is pushed first) and the called procedure must clean up the stack

Passing parameters via stack (cont.)

IMPORTANT!

NEAR procedure and calling convention is STDCALL

Ones proc near stdcall

;Parameters: 1)doubleword value, 2)doubleword - number of rightmost bits to count

;Return value: result in EAX

```
push ebp      ; save EBP
```

```
mov  ebp, esp ; save ESP to EBP
```

```
push ebx      ; save EBX
```

```
push ecx      ; save ECX
```

```
mov  ecx, [ebp+12] ;ECX=second parameter
```

```
mov  eax, [ebp+8]  ;EAX=first parameter
```

```
xor  ebx, ebx
```

testb:

```
test eax, 1
```

```
jz   skip
```

```
inc  ebx
```

skip:

```
shr  eax, 1
```

```
loop testb
```

```
mov  eax, ebx      ; copy result to EAX
```

```
pop  ecx           ; restore ECX
```

```
pop  ebx           ; restore EBX
```

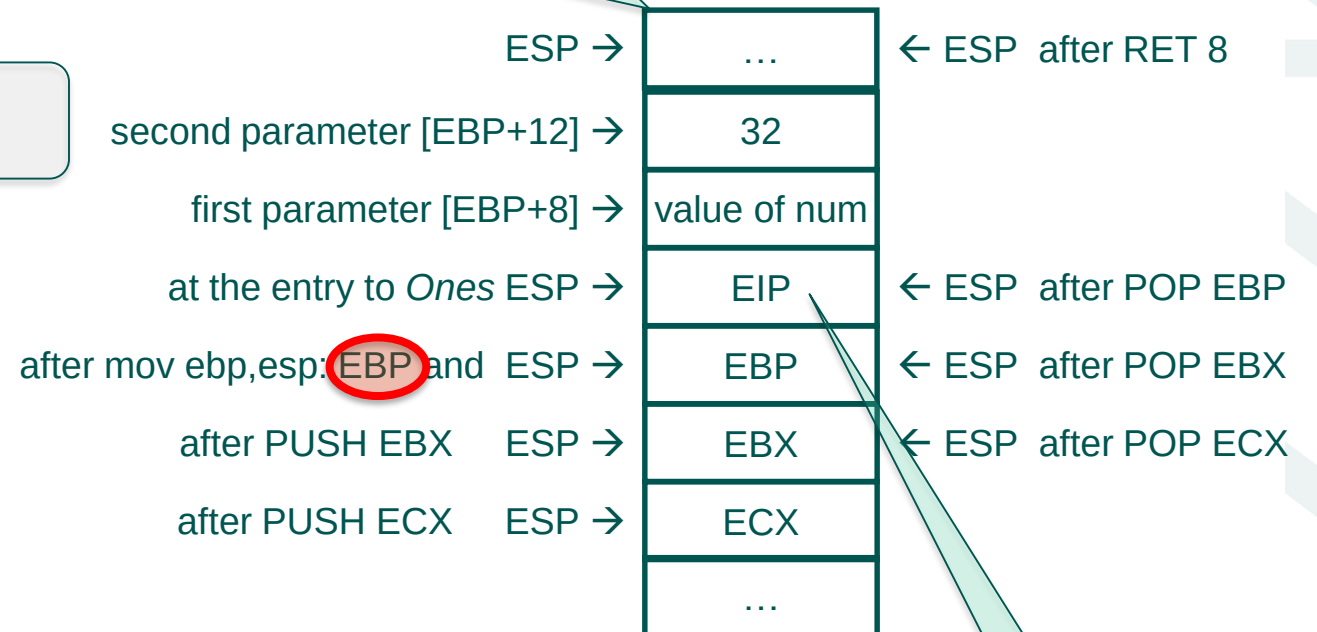
```
pop  ebp           ; restore EBP
```

```
ret  8 ;return and adjust top of the stack
```

Ones endp

After returning to the calling procedure, the top of the stack is where it was before the parameters were pushed onto the stack

Stack



NEAR procedure call pushes only EIP onto stack

In STDCALL procedure must adjust top of the stack

Thank You!

